Machine Learning in Engineering Science

Week 16: Agent Studio II: evaluation and demo rehearsal

Chi-Hua Jonny Yu / NCKU + LAiMM

Fall 2026 / 2026-12-21

Chapter 1

Agent Studio II: evaluation and demo rehearsal

Week 16: 2026-12-21, Monday 14:00–17:00.

Evaluate two supplied local workflow candidates against literal independent fixtures. This is not a benchmark of natural-language understanding, a live LLM, or real thermal safety.

Learning outcome

Compare workflow candidates against independent fixtures; retain per-case failures and make a bounded release decision.

The meeting contains three 60-minute modules: 60 minutes explanation and 120 minutes practice. The eight core exercises and two optional challenges are in-class practice, not additional weekly graded homework.

1.1 H1: Design an evaluation before the demonstration

1.1.1 Evaluate behavior, not presentation polish

A convincing demo can hide a wrong calculation or forbidden operation.

Worked reasoning. Case; expected status; numeric tolerance; safety violation; split; release gate.

Check. What does a beautiful report fail to establish?

Explanation and boundary. Correctness, provenance, boundary enforcement and performance across cases. Each needs its own evidence.

1.1.2 The case is the unit of evaluation

One fixture specifies a complete input and expected behavior.

Worked reasoning. A request containing three readings is one workflow case, not three independent successes.

Check. Why state the denominator?

Explanation and boundary. A success rate is uninterpretable without the number and definition of evaluated cases.

1.1.3 Expected answers must be independent

Write literal fixture expectations before running either candidate.

Worked reasoning. [79,80,81] C must have mean 80 and review_required under the inclusive class protocol.

Check. Why not call the reference to generate every label?

Explanation and boundary. A shared bug could make the reference and candidate agree. Independent fixtures expose correlated implementation errors.

1.1.4 Separate development and held-out cases

Use development failures to change code, then freeze before evaluation.

Worked reasoning. Named dev and held-out lists are explicit; do not shuffle repeatedly to find an attractive score.

Check. Does a held-out fixture set prove deployment validity?

Explanation and boundary. No. It bounds one local evaluation and may not represent real measurement distributions or adversaries.

1.1.5 Status correctness and numeric accuracy differ

A candidate can match the number while returning the wrong action status.

Worked reasoning. Valid cases pass only when mean, SD and status match; invalid cases pass only with the declared rejection.

Check. Why keep component failures?

Explanation and boundary. They reveal the mechanism of error and prevent a single aggregate from hiding safety or data-handling defects.

1.1.6 Numerical tolerance is a rule, not a repair

Compare absolute error to atol + rtol*abs(expected).

Worked reasoning. Expected=80, atol=1e-8, rtol=1e-10 gives limit 1.8e-8.

Check. Can increasing tolerance after a failure be legitimate?

Explanation and boundary. Only with an independently justified protocol revision and new report. Do not widen it merely to make a candidate pass.

1.1.7 An error budget is not a safety excuse

Report forbidden-action violations separately from task correctness.

Worked reasoning. One unauthorized operation remains unacceptable even if nine ordinary cases succeed.

Check. Why avoid averaging severity away?

Explanation and boundary. Different failures have different consequences. A predefined safety gate can block release regardless of mean task accuracy.

1.1.8 The baseline makes added complexity accountable

A deterministic pipeline is the first candidate, not an embarrassment.

Worked reasoning. The supplied buggy candidate uses n+1 for its mean; nonzero inputs expose it.

Check. Is this an LLM quality benchmark?

Explanation and boundary. No. It evaluates local workflow implementations. Model proposal quality would require separately collected and labeled outputs.

1.1.9 Predeclare a release rule

All required fixtures pass and zero protected side effects occur in this class gate.

Worked reasoning. No candidate is approved for real deployment; passing means eligible for the classroom demonstration only.

Check. Why include the scope in the decision?

Explanation and boundary. A finite teaching fixture suite does not validate equipment, security, clinical use or broader autonomous operation.

1.1.10 Summary: freeze what will be compared

Preserve candidates, cases, expected outputs, tolerance and release gate.

Worked reasoning. Explain which defect it could expose and why current cases might miss it.

Check. What happens next?

Explanation and boundary. Execute both candidates, retain the full failure table and rehearse a truthful demonstration.

1.2 H2: Run the harness and explain candidate failures

1.2.1 One harness must judge both candidates

Apply identical cases, labels and tolerance to each implementation.

Worked reasoning. Do not grant one candidate a different input repair rule or a more generous tolerance.

Check. What invalidates the comparison?

Explanation and boundary. Changing the evaluation procedure by candidate without an explicit reason and a separate reported protocol.

1.2.2 Return structured failures, not silent skips

A candidate exception is an observed outcome with a case ID.

Worked reasoning. Skipping failed cases can inflate the apparent success rate.

Check. Should every exception be treated as a security breach?

Explanation and boundary. No. Classify invalid input handling, implementation errors and side-effect violations separately.

1.2.3 Task 3: inspect the per-case table

Show expected status, actual status, mean error and pass flag.

Worked reasoning. A failed equality case should connect the wrong denominator to the missed review status.

Check. Why export rows as well as a chart?

Explanation and boundary. A bar chart cannot identify which fixture failed or reconstruct the mechanism.

1.2.4 A zero-mean case can conceal a bug

Changing n to n+1 leaves a zero numerator unchanged.

Worked reasoning. [0,0,0] can pass the mean check; [79,80,81] cannot pass under the buggy denominator.

Check. What kind of testing lesson is this?

Explanation and boundary. Test discriminating inputs that separate plausible defects, not merely convenient examples.

1.2.5 Measure the declared subgroup outcomes

Report valid cases, invalid cases and boundary cases separately.

Worked reasoning. A 100 percent result on one case is a count of one, not a robust population estimate.

Check. Can overlapping groups be added as a total?

Explanation and boundary. No. A boundary case may also be valid. Use the complete unique case list for overall denominators.

1.2.6 Fixed-case uncertainty is not population certainty

A small nonrandom suite estimates no universal deployment success rate.

Worked reasoning. The fixture list was designed for bugs, not sampled independently from a deployment population.

Check. What would support a population performance estimate?

Explanation and boundary. A justified sampling design, representative cases, independent units and uncertainty appropriate to that design.

1.2.7 Task 5: inspect an invalid-input success

A required rejection is a correct case outcome, not missing data.

Worked reasoning. An invalid case has no expected numeric mean. Do not fabricate zero to fit it into a numeric-error plot.

Check. What should a numeric error be for rejected inputs?

Explanation and boundary. Unavailable, with a reason. Numeric metrics should use only cases for which the quantity is defined.

1.2.8 Task 6: apply the frozen release gate

A candidate with any required fixture failure remains blocked.

Worked reasoning. All declared mandatory cases must pass; report exact failing case IDs for a blocked candidate.

Check. Is passing this gate deployment authorization?

Explanation and boundary. No. It is a local demonstration-readiness decision with explicit untested threats.

1.2.9 Rehearse evidence before screenshots

The demonstration should show one success and one expected failure.

Worked reasoning. Problem 45s; mechanism 60s; successful run 75s; failed case 60s; limits and replay 60s.

Check. Why preserve the failure in the demo?

Explanation and boundary. It shows the system boundary and evidence of control rather than implying universal success.

1.2.10 Summary: explain why a candidate failed

Name the fixture, violated rule and causal code path.

Worked reasoning. Save candidate hashes, suite hash, per-case CSV, release JSON and plots.

Check. What may you change after held-out inspection?

Explanation and boundary. Make an explicitly exploratory repair, then evaluate on a new independent set before a new final claim.

1.3 H3: Studio II: evaluate, challenge, rehearse

1.3.1 Keep the evaluator independent of the candidate

Do not repair expected labels to match generated results.

Worked reasoning. Restore 5 min; evaluation 25 min; challenge design 15 min; rehearsal and export 15 min.

Check. What is the evidence contract?

Explanation and boundary. Per-case outcomes, explicit denominators, candidate and suite identity, release gate and an untested limitation.

1.3.2 Task 1: predict which fixture exposes n+1

Select a nonzero-mean case before running the buggy candidate.

Worked reasoning. Manually compute sum/len for the selected fixture and preserve that expected value.

Check. What if the candidate accidentally passes one case?

Explanation and boundary. Report the pass honestly but inspect whether the fixture discriminates the suspected defect.

1.3.3 Task 2: freeze the release gate

Write the all-required-cases rule and zero-side-effect condition.

Worked reasoning. A later relaxed rule is a different experiment, not the same passing result.

Check. Why record gate identity with the output?

Explanation and boundary. Without it, the word pass does not identify what was checked or how strict the conditions were.

1.3.4 Task 3: inspect one failing row

Read case ID, expected status, actual status and numeric mismatch.

Worked reasoning. An incorrect low mean can prevent required review even when the output format is valid.

Check. Is schema validity enough here?

Explanation and boundary. No. It establishes structure but not the numeric and decision semantics.

1.3.5 Task 4: explain the hidden zero numerator

A zero numerator erases a denominator defect on that fixture.

Worked reasoning. Use [1,2,6] or a threshold-sensitive case and explain why the outputs differ.

Check. Does adding this after seeing failures preserve a held-out claim?

Explanation and boundary. No. Treat it as a diagnostic development probe unless a fresh independent evaluation is declared.

1.3.6 Task 5: check a required rejection

Inspect empty values and an unknown unit as separate cases.

Worked reasoning. A made-up mean on malformed input is not useful behavior under the contract.

Check. Why keep rejected inputs in the case table?

Explanation and boundary. They define the interface boundary and prevent functionality-only tests from overlooking unsafe acceptance.

1.3.7 Task 6: defend the release decision

Show the exact failed IDs or all passing required IDs.

Worked reasoning. A blocked candidate is not repaired by the success of the reference.

Check. What limitation survives a passing suite?

Explanation and boundary. Coverage is finite and synthetic. It does not establish real-world reliability or live LLM behavior.

1.3.8 Task 7: rehearse the five-minute narrative

Show inputs and actual outputs before interpreting their meaning.

Worked reasoning. Keep the full 300-second budget and identify the artifact supporting each segment.

Check. Is another five-minute submission required?

Explanation and boundary. No. This is optional in-class rehearsal. It does not add a new weekly graded video assignment.

1.3.9 Task 8: export an inspectable evaluation

Download case table, configuration, hashes, responses and plots.

Worked reasoning. A classmate counts unique cases and checks one composite pass rule from the exported rows.

Check. Why not export only aggregate accuracy?

Explanation and boundary. It loses failure identity, subgroup denominators and the distinction between numeric and interface errors.

1.3.10 Summary: improve tests before adding autonomy

Optional A adds a metamorphic relation; optional B proposes a new held-out fixture.

Worked reasoning. Choose one defect or coverage gap and design a specific independent check.

Check. What connects to Week 17?

Explanation and boundary. Package this evidence so a new process and a classmate can reconstruct the demonstration and its limits.

1.4 Evaluation reference

$$|a - e| \leq \text{atol} + \text{rtol}|e|.$$

A valid case passes only when status and all required numeric fields match. An invalid case passes by the specified rejection, with no fabricated numeric target. The class release rule requires all mandatory fixtures to pass and no protected side effects. The designed fixture suite is not a random sample from deployment; no population success interval is claimed.

1.5 Eight core practice tasks

1.5.1 Task 1: Predict which fixture exposes the denominator

Use development cases first. Calculate the expected mean manually and compare both supplied candidates.

Self-check. The zero case passes the supplied buggy mean because the numerator is zero. That does not clear the implementation.

1.5.2 Task 2: Freeze the release gate and candidate identities

Write the mandatory rule before opening the held-out result. The side-effect count below is zero by construction: these candidates have no action tools.

Self-check. Passing means classroom demonstration readiness only. A frozen gate is not a new grading policy.

1.5.3 Task 3: Run the same held-out harness for both candidates

Inspect every row before reading an aggregate. Exceptions remain failures with their case IDs.

Self-check. A required rejection has no expected numeric error. Never replace unavailable numeric values with zero.

1.5.4 Task 4: Explain why the zero case hid a defect

Contrast D-zero with the held-out inclusive boundary fixture; diagnose, but do not repair and re-report on the same held-out set.

Self-check. A numeric defect can also change a review decision. A polished report does not repair the calculation.

1.5.5 Task 5: Inspect required rejections separately

Count valid and invalid fixtures explicitly, and check the same component rules for both candidates.

Self-check. The case, not each repeated reading, is the evaluation unit. Show counts alongside every percentage.

1.5.6 Task 6: Apply the unchanged classroom release gate

Name exact failing IDs and the finite scope of each decision.

Self-check. The structural absence of side-effect tools is not a live security benchmark. No real deployment authorization is implied.

1.5.7 Task 7: Rehearse a five-minute evidence narrative

Identify the artifact supporting each segment. This is in-class practice, not an additional video submission.

Self-check. Show the actual T-boundary failure, not an invented model anecdote. Keep the original frozen evaluation intact.

1.5.8 Task 8: Export rows, not only averages

Save a CSV with defined numeric fields and explicit unavailable values for required rejection cases.

Self-check. The final ZIP includes configuration, source and all actual observations. Blank reflection fields remain draft.

1.6 Evidence handover and summary

Compare workflow candidates against independent fixtures; retain per-case failures and make a bounded release decision. Save eight predictions, eight explanations and three reflections. Preserve source, data, versions, traces and one limitation. The notebook exports a draft when fields are blank; response completeness does not establish correctness. Save the edited notebook separately if changing code.

1.7 Primary references

- Demystifying evals for AI agents: https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents.
- scikit-learn model evaluation: https://scikit-learn.org/stable/modules/model_evaluation.html.